

Assignment:- 8

ALP Program

Write 8086 ALP to find and count negative and positive number from signed array stored in memory and display magnitude of negative numbers.

```
section .data
    arr db 10, -5, -12, 7, -3, 8, 0, -9, 4, -6
    len equ 10
    pos_count db 0
    neg_count db 0
    msg_pos db "Positive count: ", 0
    msg_neg db "Negative count: ", 0
    newline db 10
```

```
section .bss
    buffer resb 4    ; for integer to string
                    conversion
```

```
section .text
    global _start
```

```
_start:
    mov rcx, len
    lea rsi, [rel arr]
```

```
loop_start:
    mov al, [rsi]
    cmp al, 0
    jl negative
    inc byte [pos_count]
    jmp next
```

```
negative:
    inc byte [neg_count]
```

```
next:
    inc rsi
```

```
loop loop_start
```

```
; print "Positive count: "
mov rsi, msg_pos
call print_string
movzx rax, byte [pos_count]
call print_number
```

```
; newline
mov rsi, newline
call print_string
```

```
; print "Negative count: "
mov rsi, msg_neg
call print_string
movzx rax, byte [neg_count]
call print_number
```

```
; newline
mov rsi, newline
call print_string
```

```
; exit
mov rax, 60
xor rdi, rdi
syscall
```

```
; -----
; print_string (rsi = address)
print_string:
    push rax
    push rdi
```

```

push rdx
mov rax, 1
mov rdi, 1
mov rdx, 0
count_loop:
    cmp byte [rsi+rdx], 0
    je print_it
    inc rdx
    jmp count_loop
print_it:
    syscall
    pop rdx
    pop rdi
    pop rax
    ret

```

; -----

```

; print_number (rax = number)
print_number:
    mov rcx, buffer+3
    mov byte [rcx], 0
convert_loop:
    dec rcx
    xor rdx, rdx
    mov rbx, 10
    div rbx
    add dl, '0'
    mov [rcx], dl
    test rax, rax
    jnz convert_loop
    mov rsi, rcx
    call print_string
    ret

```

Output:

The screenshot shows a terminal window with two panes. The left pane displays assembly code for a program that counts positive and negative numbers in an array. The right pane shows the program's output, which matches the expected results.

```

1 section .data
2     arr db 10, -5, -12, 7, -3, 8, 0, -9, 4, -6
3     len equ 10
4     pos_count db 0
5     neg_count db 0
6     msg_pos db "Positive count: ", 0
7     msg_neg db "Negative count: ", 0
8     newline db 10
9
10 section .bss
11     buffer resb 4 ; for integer to string conversion
12
13 section .text
14     global _start
15
16 _start:
17     mov rcx, len
18     lea rsi, [rel arr]
19
20 loop_start:
21     mov al, [rsi]
22     cmp al, 0
23     jl negative
24     inc byte [pos_count]
25     jmp next
26
27 negative:
28     inc byte [neg_count]
29     jmp next
30
31 next:
32     lea rsi, [rel arr]
33     add rsi, rcx
34     dec rcx
35     cmp rcx, 0
36     jne loop_start
37
38     mov rax, 0
39     syscall

```

Program input

Output

```

Positive count: 5
Negative count: 5

[Execution complete with exit code 0]

```